Data Structures and Algorithms

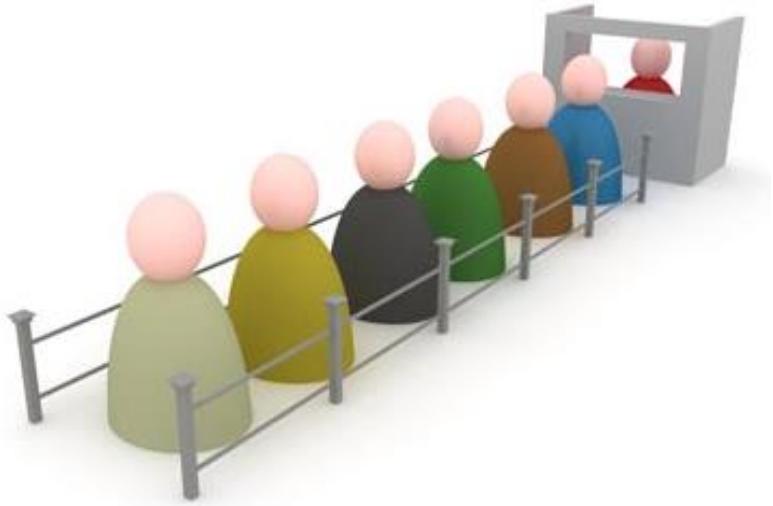
Instructor: Dr. Hao Qin（秦浩）

E-mail: hao.qin@scupi.cn

My Office: Room 514, SCUPI (North)

Lecture #4

- Stacks
- Queues

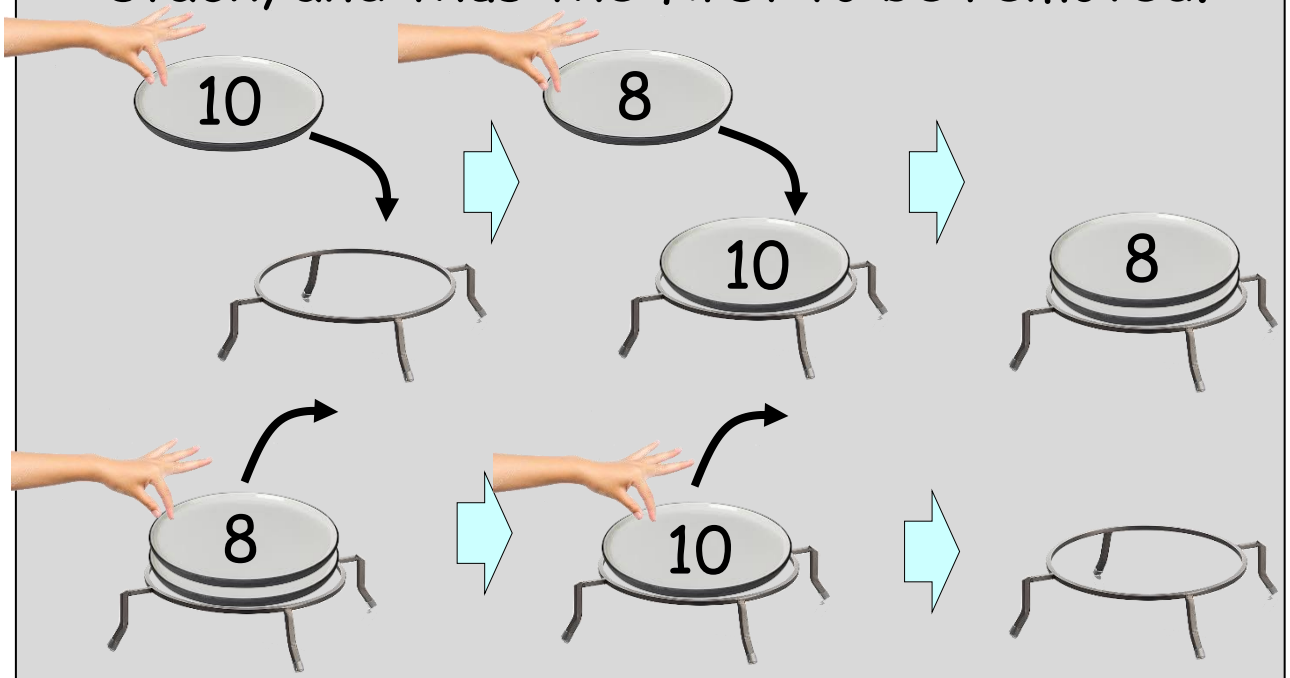


Stacks

What's the big picture?

A stack is a data structure that resembles a stack of plates at a buffet.

Just like a stack of plates, the last plate/value you add is at the top of the stack, and thus the first to be removed.



The last-added/first-removed aspect of stacks can help solve many hard problems!

Uses:

Use stacks to find a path through a maze, "undo" in a word processor, and evaluate math expressions.

The Stack: A Useful ADT

A stack is an ADT that holds a collection of items (like ints) where the elements are always added to one end.

Just like a stack of plates, the last item pushed onto the top of a stack is the first item to be removed.

Stack operations:

- put something on top of the stack (PUSH)
- remove the top item (POP)
- look at the top item, without removing it
- check to see if the stack is empty

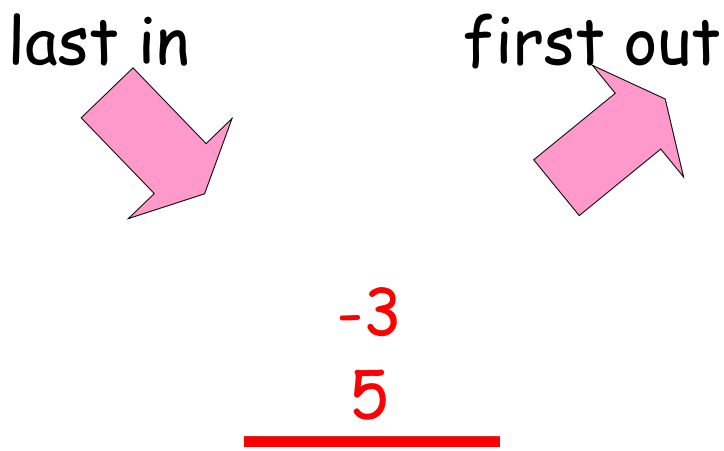
We can have a stack of any type of variable we like:
ints, Squares, floats, strings, etc.

The Stack

Note: The stack is called a Last-In-First-Out data structure.
Can you figure out why?

I can…

Push 5 on the stack.
Push -3 on the stack.
Push 9 on the stack.
Pop the top of the stack.
Look at the stack's top value.
Push 4 on the stack.
Pop the top of the stack
Pop the top of the stack
Look at the stack's top value.
Pop the top of the stack



Note: You can only access the top item of the stack, since the other items are covered.

Stacks

```
class Stack   // stack of ints
{
public:
   Stack();        // c'tor
   void push(int i);
   int pop();
   bool is_empty(void);
   int peek_top();
private:
   ...
};
```

```
int main(void)
{
    Stack is;

    is.push(10);
    is.push(20);

    ...
}
```

Question:
What type of data structure can we use to implement our stack?

Answer:
How about an array and a counter variable to track where the top of the stack is?

Implemer

```
const int SIZE  = 100;
class Stack
{
public:
    Stack() { m_top = 0;
    void push(int val) {
        if (m_top >= SIZ
        m_stack[m_top] = val;
        m_top += 1;
    }

    int pop()
        if (m_top == 0) exit(
        m_top -= 1;
        return m_stack[m_top];
    }
    ...
private:
    int m_stack[SIZ
    int m_top;
};
```

To initialize our stack,

ever

stack!

, let's

ens.

Update the location where our next item should be

Since m_top points to where our

Extract the value from the top of the stack and return it to the user.

Let's us

This st

maximu

keep track of where the next item should be added to the stack.

Stacks

```
const int SIZE  =
class Sta
{
public:
  Stac
  void
    i
    m
    m
  }

  int po
    if                    / underflow
    m_top
    return m_stack[m_top];
  }

  ...
private:
    int m_stack[SIZE];
    int m_top;
};
```

The first item we push will be placed in

Currently, our m_top points to the next open slot in the stack…

But we want to return the top item already pushed on the stack.

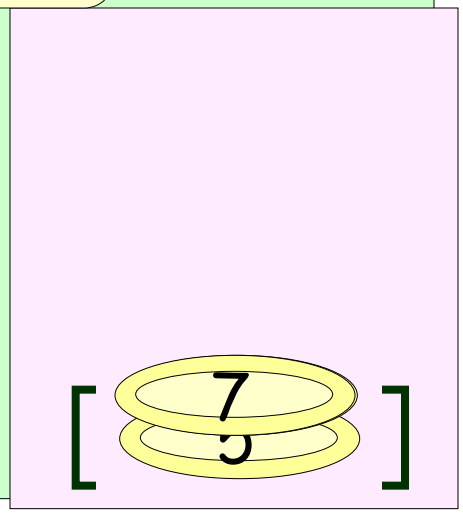
So first we must decrement our m_top variable…

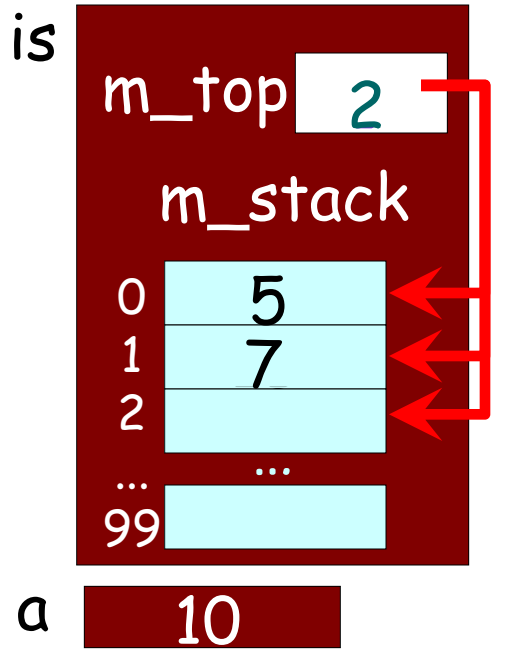
re …_stack[1]

So we return 10

```
int main(void)
{
  Stack is;
  int a;

  is.push(5);
  is.push(10);
  a = is.pop();
  cout << a;
  is.push(7);
}
```





```
const int SIZE  = 100;
class Stack
{
public:
    Stack() { m_top = 0; }
    void push(int val) {
        if (m_top >= SIZE) exit(-1); // overflow
        m_stack[m_top] = val;
        m_top += 1;
    }

    int pop() {
        if (m_top == 0) exit(-1); // underflow
        m_top -= 1;
        return m_stack[m_top];
    }

    ...
private:
     int m_stack[SIZE];
     int m_top;
};
```

Always Remember:

When we push, we:

A. Store the new item in m_stack[m_top]
B. Post-increment our m_top variable
(post means we do the increment after storing)

Always Remember:

When we pop, we:

A. Pre-decrement our m_top variable
B. Return the item in m_stack[m_top]
(pre means we do the decrement before returning)

S

Stacks are
one for you

Here's the

std::st

For

std::stack<string> stackOfStrings;
std::stack<double> stackOfDoubles;

And as with cin and cout, you can remove the std:: prefix if you add a using namespace std; command!

```
#include <ios
#include <sta
using namespace std;
int main()
{
        stack<int>  istack; // stack of ints

  istack.push(10);          //
  istack.push(20);

  cout << istack.top();   //
  istack.pop();

  if (istack.empty() == fa
    cout << istack.size()
}
```

So to get the top item's value, before popping it, use the top() method!

Note: The STL pop() command simply throws away the top item from the stack...
but it doesn't return it.

Stack Challenge

Show the resulting stack after the following program runs:

```
#include <iostream>
#include <stack>
using namespace std;

int main()
{
  stack<int>  istack;       // stack of ints

  istack.push(6);
  for (int i=0;i<2;i++)
  {
     int n = istack.top();
     istack.pop();
     istack.push(i);
     istack.push(n*2);
  }
}
```

Stack Challenge

Show the resulting stack after the following program runs:

```
#include <iostream>
#include <stack>
using namespace std;

int main()
{
  stack<int>  istack;       // stack of ints

  istack.push(6);
  for (int i=0;i<2;i++)
    {
    int n = istack.top();
    istack.pop();
    istack.push(i);
    istack.push(n*2);
    }
}
```

A Stack... in your CPU!

Did you know that every CPU has a built-in stack used to hold local variables and f


```
void bar(int b)
{
  cout << b << endl;
}

void foo(int a)
{
  cout << a << endl;
  bar(10);
}

int main(void)
{
  int x = 5;
  foo( 5 );
}
```

When you pass a value to a function, the CPU pushes that value onto a *stack* in the computer's memory.

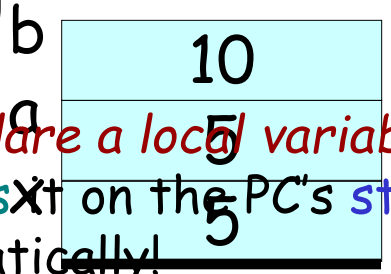
Local variables are stored on the computer's built-in stack!

tion,
stack

rns,

off
y.

returns, the values
tack and go away.



u *declare a local variable,*
your program pushes it on the PC's stack
automatically!

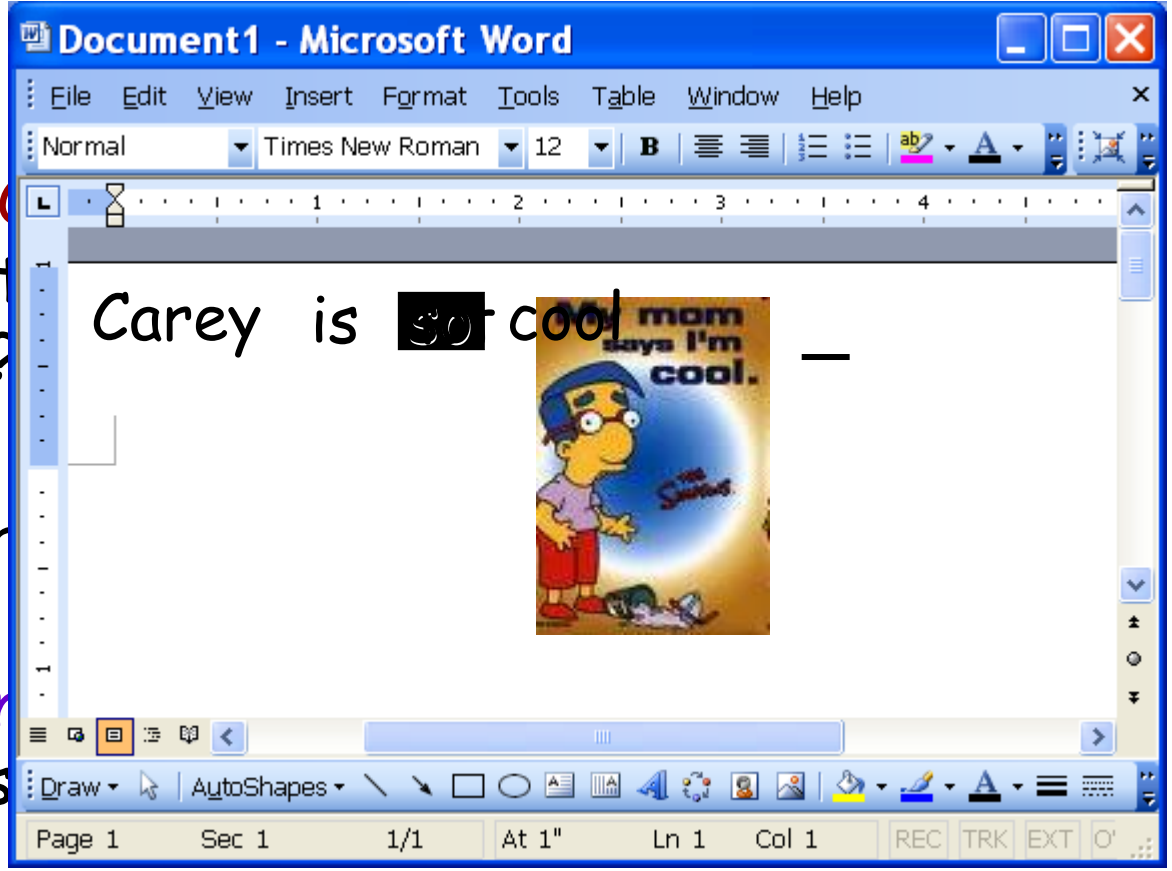
When the user hits the undo button…

So how does the UNDO feature of your favorite word processor work?

The word processor pops the top item off the stack and removes it from the document!

It uses a stack, of course!

Every time you type a new word, it's added to the stack!



Every time you cut-and-paste an image into your doc it's added to the stack!

In this way, the word processor can track the last X things that you did and properly undo them!

And even when you delete text or pictures, this is tracked on a stack!

"so" → "not"
"cool"
"so"
"is"
"Carey"

undo stack

Postfix Expression Evaluation

Most people are used to infix notation, where the operator is in-

Postfix notation is
- here the o

As we'll see, postfix expressions have no such ambiguity!

Infix	Postf
	15 6 +

If you've ever used an HP calculator, you've used postfix notation!

Is that (5+10) * 3
or
5 + (10 * 3)
To understand infix expressions, the computer has to be equipped with precedence rules!

ıs c

essions, because they're unan

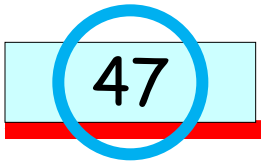
nfix expression example: 5 + 10 * 3

Postfix Evaluation Algorithm

Inputs: postfix expression string
Output: number representing answer
Private data: a stack

7 6 * 5 +

1. Start with the left-most token.
2. If the token is a number:
   a. Push it onto the stack
3. Else if the token is an operator:
   a. Pop the top value into a variable called v2, and the second-to-top value into v1.
   b. Apply operator to v1 and v2 (e.g., v1 / v2)
   c. Push the result of the operation on the stack
4. If there are more tokens, advance to the next token and go back to step #2
5. After all tokens have been processed, the top # on the stack is the answer!



... // we'll see this in a bit!

Infix to Postfix Conversion

Stacks can also be used to conver
postfix express

For example,

From: (3 + 5) * (4 + 3 / 2)
To: 3 5 + 4 3 2 / + * 5 -

Or

From: 3 + 6 * 7 * 8 - 3
To: 3 6 7 * 8 * + 3 -

Since people are more used to infix notation…

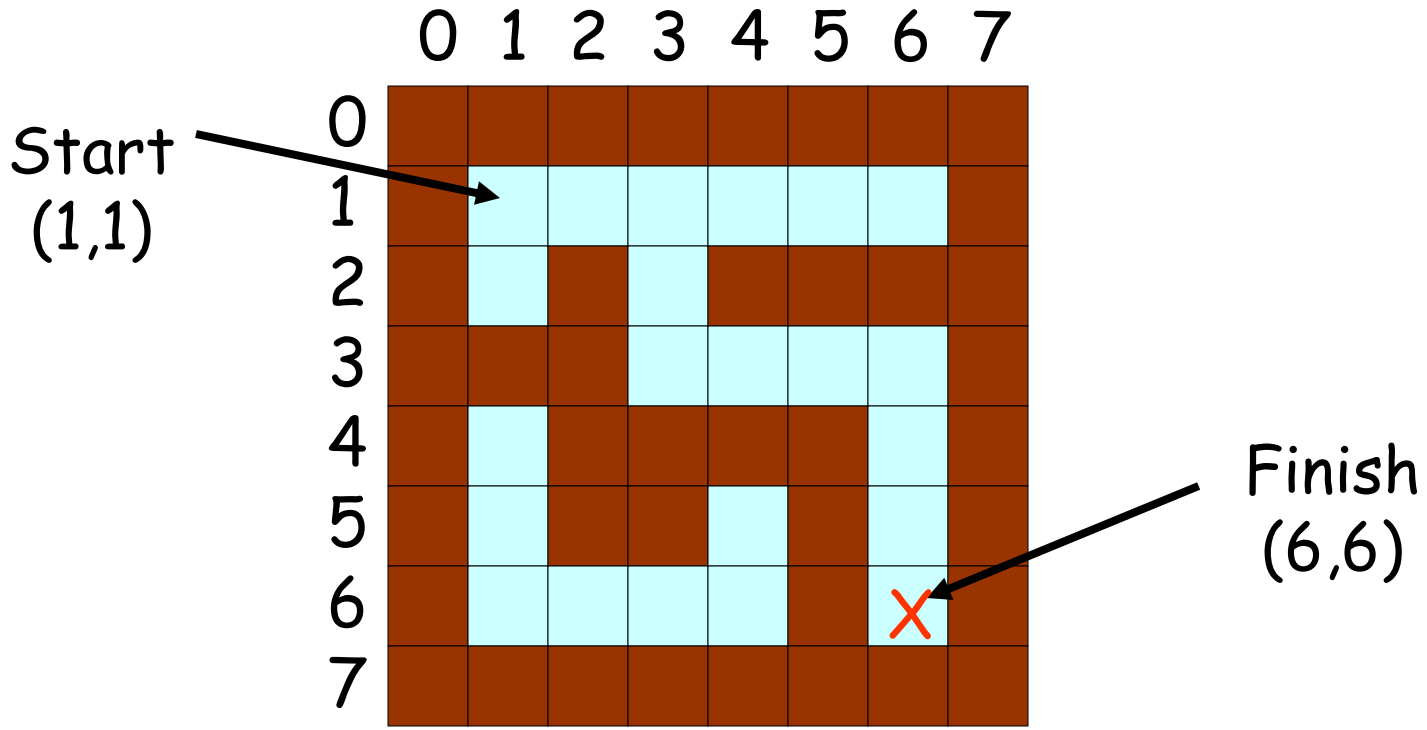
You can let the user type in an infix expression…

And then convert it into a postfix expression.

Finally, you can use the postfix evaluation alg (that we just learned) to compute the value of the expression.

Solving a Maze with a Stack!

We can also use a stack to determine if a maze is solvable:



Solving a Maze with a Stack!

Inputs: 10x10 Maze in a 2D array,
Starting point (sx,sy)
Ending point (ex,ey)
Output: TRUE if the maze can be solved, FALSE otherwise
Private data: a stack of *points*

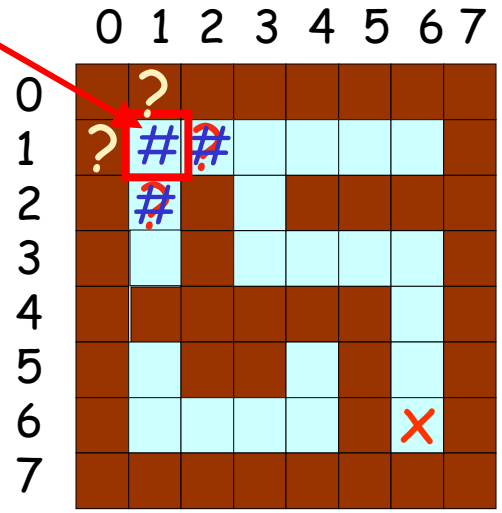
```
class Point
{
public:
   Point(int x, int y);
   int getx() const;
   int gety() const;
private:
   int m_x, m_y;
};
```

```
class Stack
{
public:
    Stack();      // c'tor
    void push(Point &p);
    Point pop();
    ...
private:
    ...
};
```

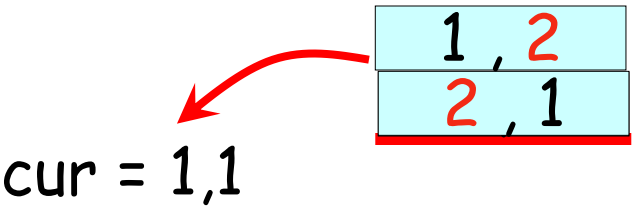
Solving a Maze with a Stack!

1. PUSH starting point onto the stack.
2. Mark the starting point as "discovered."
3. While the stack is not empty:
   - A. POP the stack to get the next point to explore and store it into a variable.
   - B. If we reached the endpoint, DONE! Otherwise…
   - C. If slot to the WEST is open & is undiscovered
     Mark (curx-1,cury) as "discovered"
     PUSH (curx-1,cury) on stack.
   - D. If slot to the EAST is open & is undiscovered
     - Mark (curx+1,cury) as "discovered"
     - PUSH (curx+1,cury) on stack.
   - E. If slot to the NORTH is open & is undiscovered
     Mark (curx,cury-1) as "discovered"
     PUSH (curx,cury-1) on stack.
   - F. If slot to the SOUTH is open & is undiscovered
     - Mark (curx,cury+1) as "discovered"
     - PUSH (curx,cury+1) on stack.
4. If the stack is empty and we haven't reached our goal position, then the maze is unsolvable.

sx,sy = 1,1

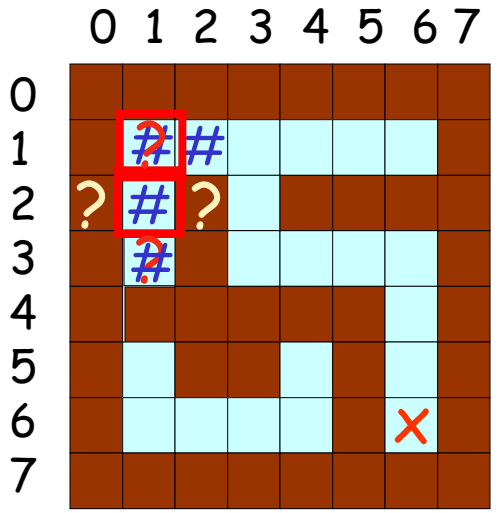


1,1 == 6,6?
Not yet!

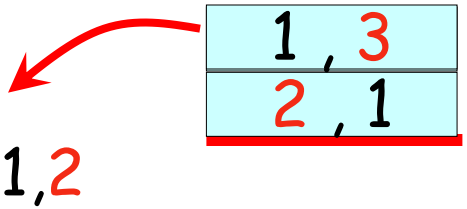


Solving a Maze with a Stack!

1. PUSH starting point onto the stack.
2. Mark the starting point as "discovered."
3. While the stack is not empty:
   - A. POP the top point off the stack into a variable.
   - B. If we're at the endpoint, DONE! Otherwise…
   - C. If slot to the WEST is open & is undiscovered
     Mark (curx-1,cury) as "discovered"
     PUSH (curx-1,cury) on stack.
   - D. If slot to the EAST is open & is undiscovered
     Mark (curx+1,cury) as "discovered"
     PUSH (curx+1,cury) on stack.
   - E. If slot to the NORTH is open & is undiscovered
     Mark (curx,cury-1) as "discovered"
     PUSH (curx,cury-1) on stack.
   - F. If slot to the SOUTH is open & is undiscovered
     - Mark (curx,cury+1) as "discovered"
     - PUSH (curx,cury+1) on stack.
4. If the stack is empty and we haven't reached our goal position, then the maze is unsolvable.



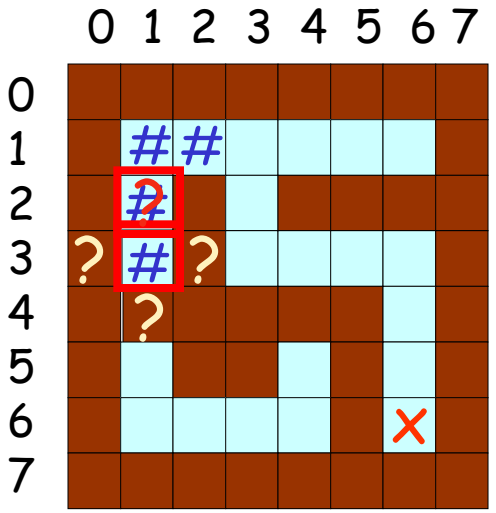
1,2 == 6,6?
Not yet!



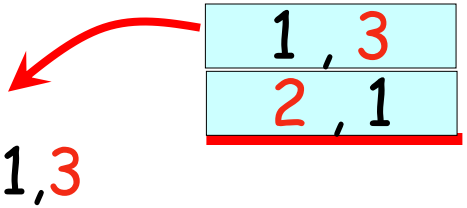
cur = 1,2

Solving a Maze with a Stack!

1. PUSH starting point onto the stack.
2. Mark the starting point as "discovered."
3. While the stack is not empty:
   - A. POP the top point off the stack into a variable.
   - B. If we're at the endpoint, DONE! Otherwise…
   - C. If slot to the WEST is open & is undiscovered
     Mark (curx-1,cury) as "discovered"
     PUSH (curx-1,cury) on stack.
   - D. If slot to the EAST is open & is undiscovered
     Mark (curx+1,cury) as "discovered"
     PUSH (curx+1,cury) on stack.
   - E. If slot to the NORTH is open & is undiscovered
     Mark (curx,cury-1) as "discovered"
     PUSH (curx,cury-1) on stack.
   - F. If slot to the SOUTH is open & is undiscovered
     Mark (curx,cury+1) as "discovered"
     PUSH (curx,cury+1) on stack.
4. If the stack is empty and we haven't reached our goal position, then the maze is unsolvable.



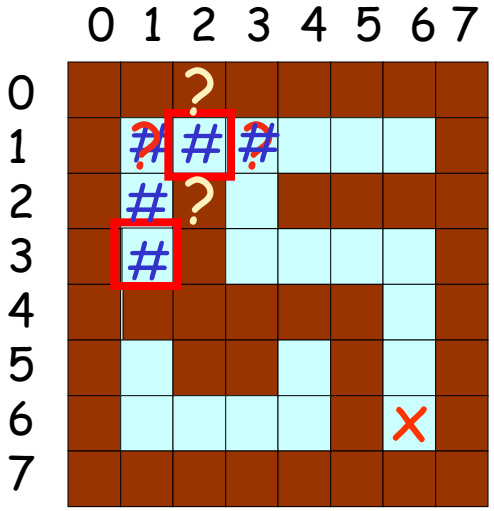
1,3 == 6,6?
Not yet!



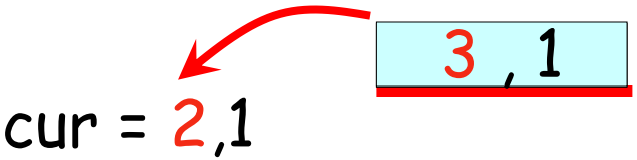
cur = 1,3

Solving a Maze with a Stack!

1. PUSH starting point onto the stack.
2. Mark the starting point as "discovered."
3. While the stack is not empty:
   - A. POP the top point off the stack into a variable.
   - B. If we're at the endpoint, DONE! Otherwise…
   - C. If slot to the WEST is open & is undiscovered
     Mark (curx-1,cury) as "discovered"
     PUSH (curx-1,cury) on stack.
   - D. If slot to the EAST is open & is undiscovered
     - Mark (curx+1,cury) as "discovered"
     - PUSH (curx+1,cury) on stack.
   - E. If slot to the NORTH is open & is undiscovered
     Mark (curx,cury-1) as "discovered"
     PUSH (curx,cury-1) on stack.
   - F. If slot to the SOUTH is open & is undiscovered
     Mark (curx,cury+1) as "discovered"
     PUSH (curx,cury+1) on stack.
4. If the stack is empty and we haven't reached our goal position, then the maze is unsolvable.



2,1 == 6,6?
Not yet!



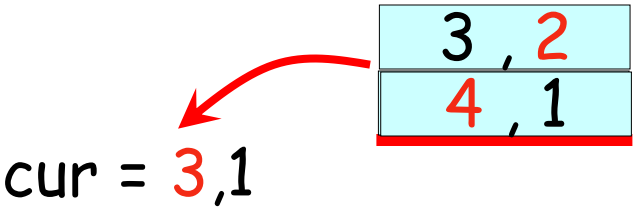
Solving a Maze with a Stack!

1. PUSH starting point onto the stack.
2. Mark the starting point as "discovered."
3. While the stack is not empty:
   - A. POP the top point off the stack into a variable.
   - B. If we're at the endpoint, DONE! Other
   - C. If slot to the WEST is open
     Mark (curx-1,cur
     PUSH (
   - open & is undiscovered
     y-1) as "discovered"
     SH (curx,cury-1) on stack.
   - F. If slot to the SOUTH is open & is undiscovered
     - Mark (curx,cury+1) as "discovered"
     - PUSH (curx,cury+1) on stack.
4. If the stack is empty and we haven't reached our goal position, then the maze is unsolvable.

3,1 == 6,6?
Not yet!

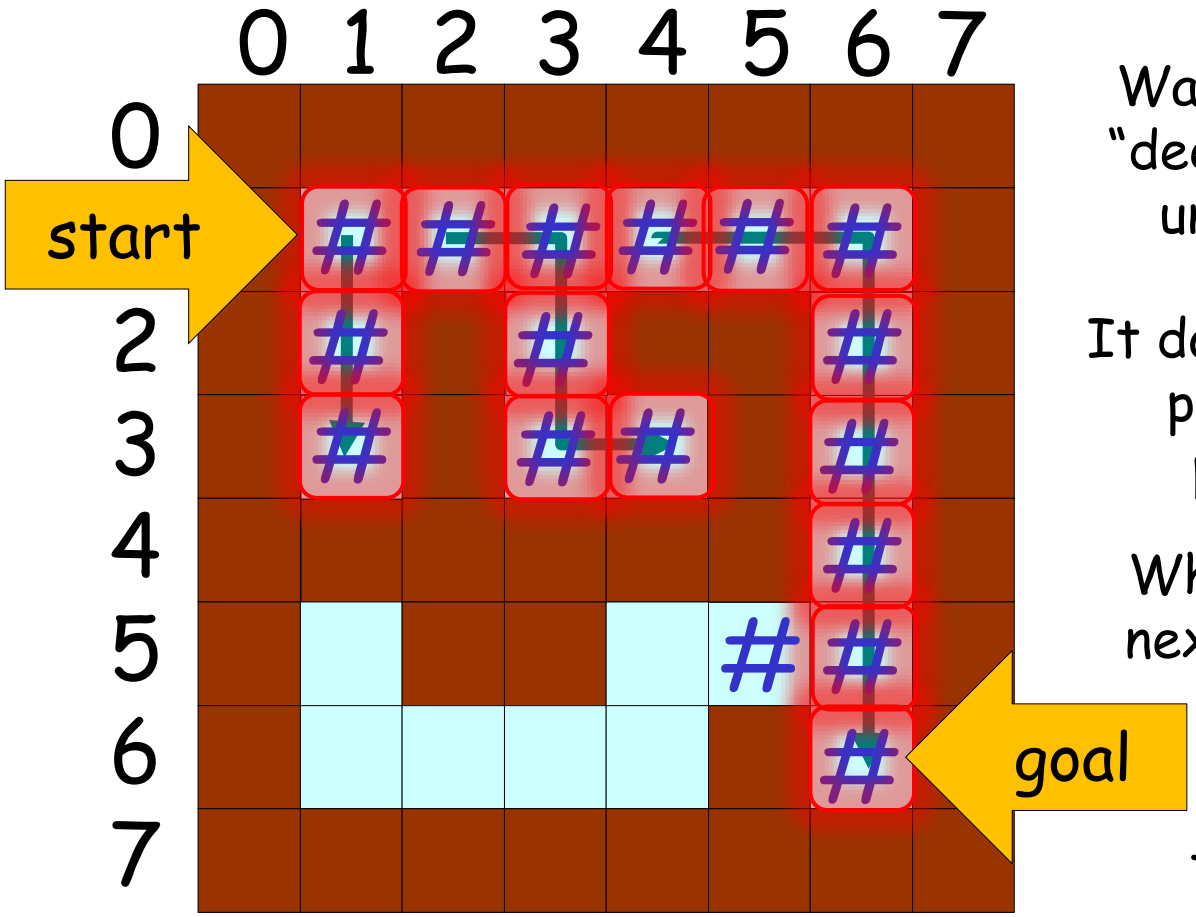
Eventually, we'll find the solution to the maze, or our stack will empty out, indicating that there is no solution!

This searching algorithm is called a "depth-first search."



Solving a Maze with a Stack

Depth-first Search Visualization



Watch how our search goes "deep" in the same direction until it hits a dead end…

It does this because it always processes the last item pushed on the stack…

Which happens to be right next to the current square being processed.

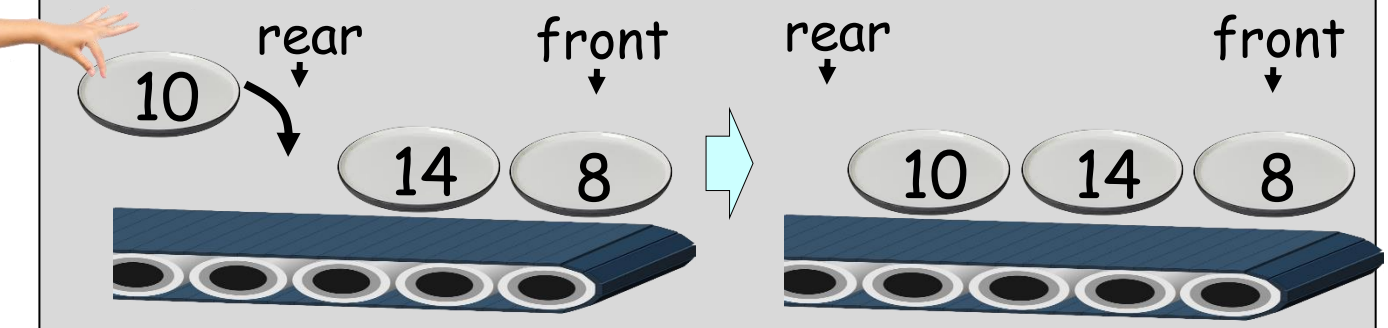
This is why it's called a "depth-first" search.

Queues

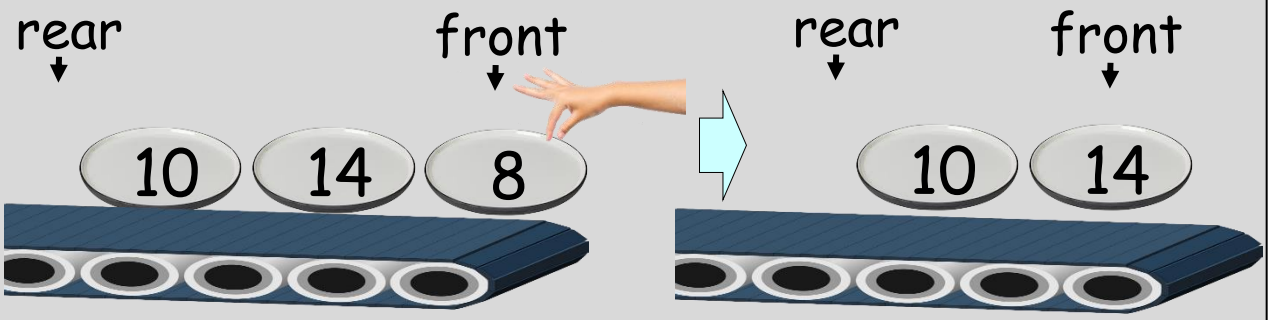
What's the big picture?

A queue is a super-useful* data structure that resembles a conveyer belt of dishes.

Dishes are added to the rear of the queue...



and dishes are extracted from its front...



The first-in/first-out property of queues is opposite of the last-in-first-out property of stacks.

Uses:

Use queues for vehicle navigation, implementing twitter feeds, queueing songs on Spotify, etc.

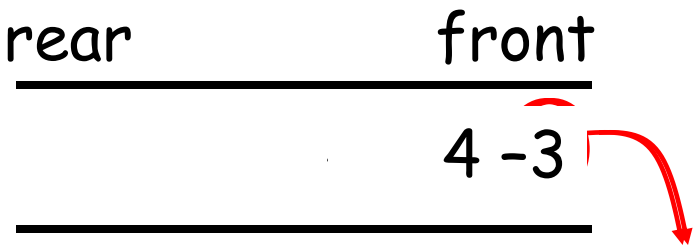
Another ADT: The Queue

The queue is another ADT that is just a like a line at the store or at the bank.

The first person in line is the first person out of line and served.

This is called a FIFO data structure:
FIRST IN, FIRST OUT.

Every queue has a *front* and a *rear*. You enqueue items at the *rear* and dequeue from the front.



What data structures could you use to implement a queue?

The Queue Interface

void enqueue(int a):
Inserts an item on the rear of the queue

int dequeue():
Removes and returns the top item from the front of the queue

bool isEmpty():
Determines if the queue is empty

int size():
Determines the # of items in the queu

int getFront():
Gives the value of the top item on the
without removing it like dequeue

Like a Stack, we can have queues of any type of data! Queues of strings, Points, Nerds, ints, etc!

Common Uses for Queues

Often, data flows from the Internet faster than the computer can use it. We use a queue to hold the data until the browser is ready to display it...

Every time your computer receives a character, it enqueues it:

```
internetQueue.enqueue(c);
```

Every time your Internet browser is ready to get and display new data, it looks in the queue:

```
while (internetQueue.isEmpty() == false)
{
    char ch = internetQueue.dequeue();

    cout << ch;  // display web page...
}
```

Common Uses for Queues

You can also use queues to search through mazes!

If you use a queue instead of a stack in our searching algorithm, it will search the maze in a different order…

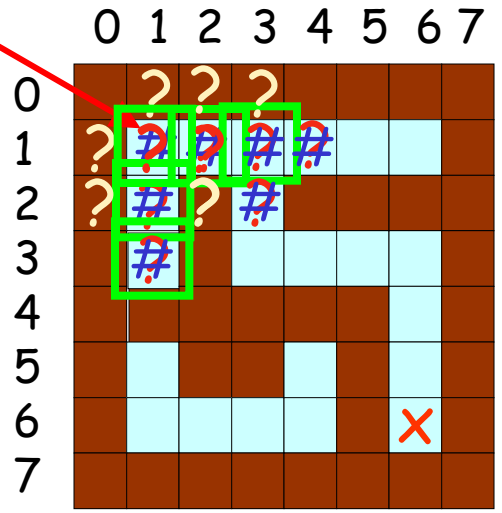
Instead of always exploring the last x,y location pushed on top of the stack first…

The new algorithm explores the oldest x,y location inserted into the queue first.

Solving a Maze with a Queue!

(AKA Breadth-first Search)

sx,sy = 1,1



1. Insert starting point onto the queue.
2. Mark the starting point as "discovered."
3. While the queue is not empty:
   - A. Remove the front point from the queue.
   - B. If we reached the endpoint, DONE! Otherwise…
   - C. If slot to the WEST is open & is undiscovered
     Mark (curx-1,cury) as "discovered"
     INSERT (curx-1,cury) on queue.
   - D. If slot to the EAST is open & is undiscovered
     - Mark (curx+1,cury) as "discovered"
     - INSERT (curx+1,cury) on queue.
   - E. If slot to the NORTH is open & is undiscovered
     Mark (curx,cury-1) as "discovered"
     INSERT (curx,cury-1) on queue.
   - F. If slot to the SOUTH is open & is undiscovered
     - Mark (curx,cury+1) as "discovered"
     - INSERT (curx,cury+1) on queue.
4. If the queue is empty and we haven't reached our goal position, then the maze is unsolvable.

And so on…

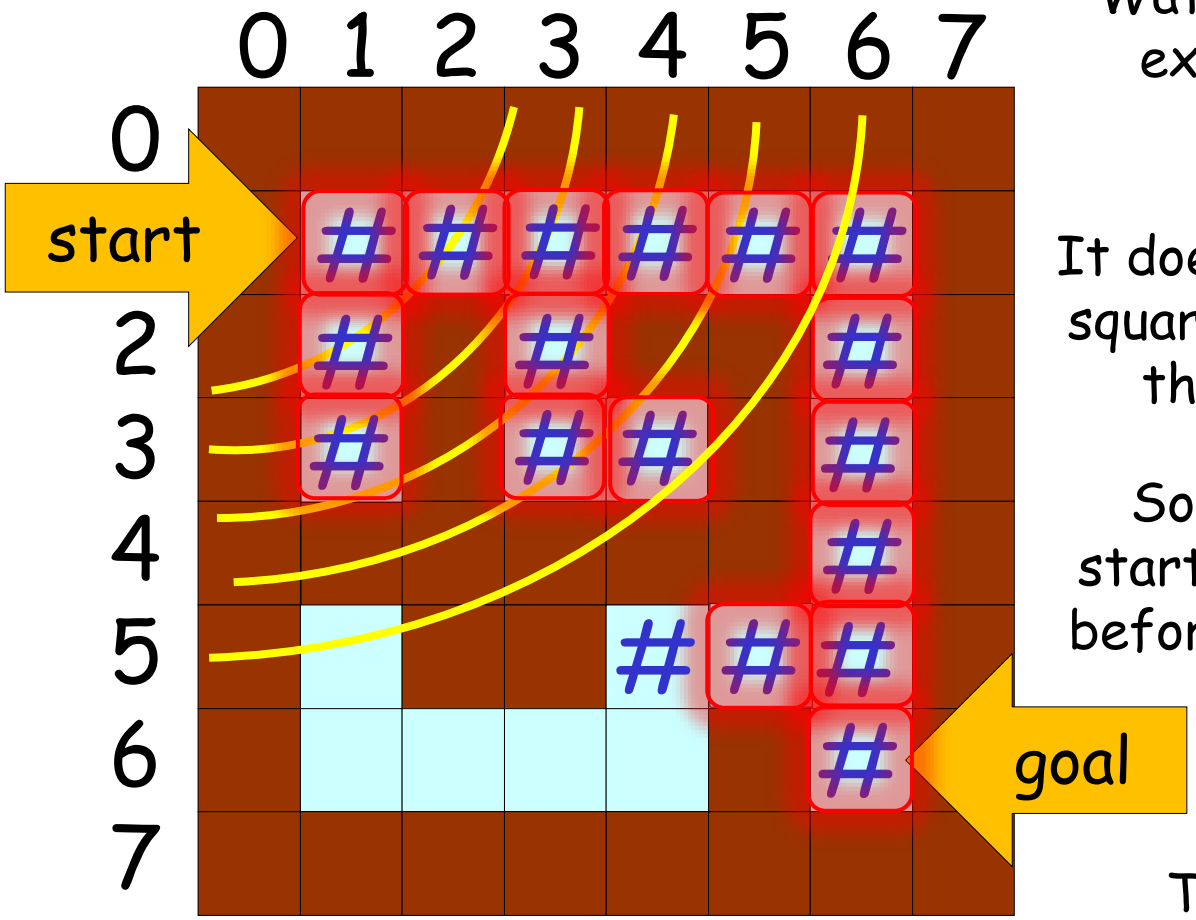
curx,cury=

rear front

(3,2) (2,1)

Solving a Maze with a Queue

Breadth-first Search Visualization



Watch how the squares we explore expand out like ripples in a pond…

It does this because each new square to explore is added to the END of the queue…

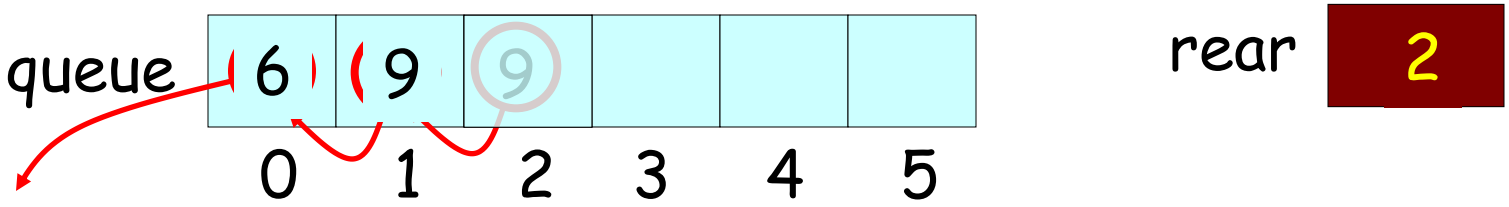
So squares closer to the starting square are explored before squares further away.

This is why it's called a "breadth-first" search.

Queue Implementations

We can use an array and an integer to represent a queue:

```
int queue[6], rear = 0;
```



- Every time you insert an item, place it in the rear slot of the array and increment the rear count
- Every time you dequeue an item, move all of the items forward in the array and decrement the rear count.

What's the problem with the array-based implementation?

If we have N items in the queue, what is the cost of:
(1) inserting a new item, (2) dequeuing an item

Queue Implementations

We can also use a linked list to represent a queue:

- Every time you insert an item, add a new node to the end of the linked list.
- Every time you dequeue an item, take it from the head of the linked list and then delete the head node.

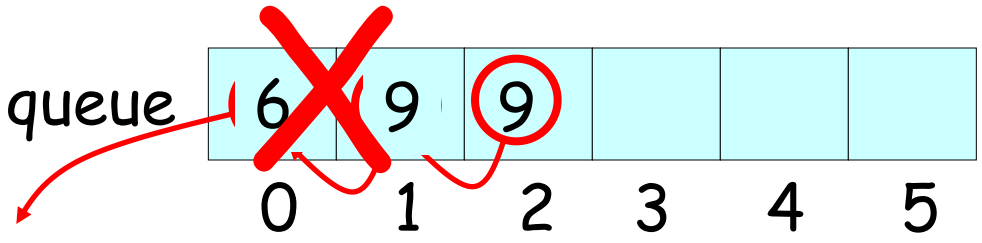
Of course, you'll want to make sure you have both head and tail pointers…

or your linked-list based queue will be really inefficient!

The Circular Queue

The circular queue is a clever type of array-based queue.

Unlike our previous array-based queue, we never need to shift items with the circular queue!



Let's see how it works!

A Queue in the STL!

The people who wrote the Standard Template Library also built a queue class for you:

```
#include <iostream>
#include <queue>

int main()
{
        std::queue<int>  iqueue;   // queue of ints

        iqueue.push(10);           // add item to rear
        iqueue.push(20);
        cout << iqueue.front();    // view front item
        iqueue.pop();         // discard front item
        if (iqueue.empty() == false)
          cout << iqueue.size();
}
```